

GENESIS 2.5: opt-in OpenCL and CUDA acceleration for the GENESIS/PGENESIS compartmental neural simulator

Karol Chlasta^{a,*}, Grzegorz Marcin Wójcik^b

^a*Kozminski University, ul. Jagiellońska 57/59, 03-301 Warsaw, Poland; WarsawIQ, Al.*

Jana Pawła II 43A/37B, 01-001 Warsaw, Poland

^b*Maria Curie-Skłodowska University, ul. Akademicka 9, 20-033 Lublin, Poland*

Abstract

GENESIS is a long-established compartmental neural simulator. PGENESIS extends it to MPI execution. We release GENESIS 2.5, an accelerator-enabled extension of GENESIS 2.4/PGENESIS 2.4, adding opt-in OpenCL and CUDA backends for the Hodgkin–Huxley channel-update path while keeping every existing CPU, MPI, model, and SLI-script workflow intact. A batched multi-step GPU kernel reaches up to $\sim 18\times$ wall-clock speedup on an integrated AMD Radeon 890M and $5\text{--}132\times$ on an NVIDIA RTX 4090, matching the fp64 CPU to $\sim 10^{-7}$ V. This multiloop kernel is exact only for single-compartment (isopotential) neurons. We additionally implement and validate `hines_tree_eliminate`, a GPU tridiagonal (Hines) elimination kernel extending acceleration to real, axially-coupled multi-compartment dendritic trees. One GPU thread per neuron runs the identical elimination the CPU solver does. Measured on UMCS cluster NVIDIA A40/A100 GPUs (10 replicates), it reaches $37.3\times$ (A40) and $82.1\times$ (A100) step-phase speedup at $N = 50\,000$ neurons $\times$ 16 compartments, still rising with N . This kernel accelerates population-scale simulation, parallelizing across neurons rather than within a single tree. Pushing this population toward the Blue Brain Project’s $\sim 31\,000$ -neuron microcircuit scale [9] surfaced a latent, GPU-independent $O(n^2)$ model-construction bottleneck. Fixing it restores linear scaling: a 1.7-million-compartment, $N = 100\,000$ population now builds in 51.5 ± 0.6 s, down from not finishing at all, a correctness-preserving fix benefit-

*Corresponding author.

Email addresses: `karol@chlasta.pl` (Karol Chlasta), `gmwojcik@umcs.edu.pl` (Grzegorz Marcin Wójcik)

ing any GENESIS/PGENESIS user, independent of accelerator use. We also fix three independent, previously unnoticed Hines-solver defects that silently suppressed `inject`-driven voltage transients in any multi-neuron `hsolve` configuration.

Keywords: GENESIS, PGENESIS, OpenCL, CUDA, GPU acceleration, compartmental neural simulation, Hines solver

Required Metadata

Current code version

Main text

1. Motivation and significance

Compartmental neural simulators remain essential for biophysically detailed models where channel kinetics and dendritic geometry cannot be abstracted away. GENESIS [1] is one of the longest-maintained simulators in this class, and PGENESIS extends it to MPI-distributed execution; both are still used in production modeling workflows, scripted via GENESIS’s SLI language, independent of newer simulator ecosystems. Modern workstations increasingly ship capable integrated GPUs, but GENESIS has no accelerator backend: every channel-kinetics update runs on the CPU regardless of available hardware, even though channel updates (per-compartment gating-variable integration) are embarrassingly parallel and GPU-friendly. Existing GENESIS/PGENESIS users have no path to accelerator support without abandoning their models and SLI scripts.

GENESIS 2.5 closes this gap: GENESIS 2.4/PGENESIS 2.4 plus an OpenCL backend for the `hsolve` channel-update path, and a CUDA backend on the same interface (a faithful fp32 port, validated on an NVIDIA RTX 4090 and UMCS cluster NVIDIA A100/A40 nodes; Section 3). No existing CPU, MPI, model, or SLI-script workflow needs to change. While validating this backend we additionally found and fixed three independent solver defects in `hines_solve.c/hines_init.c` affecting any multi-neuron `hsolve` configuration: current injection (`inject`) silently failed to produce a voltage response prior to the fix, which would have invalidated any dynamics-comparing benchmark and may have affected unrelated multi-neuron `hsolve` models elsewhere.

We name this release “2.5” rather than “3.0” deliberately: “GENESIS 3” [2] produced independent successor lines (Neurospaces/Heccer [3],

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v2.5 (2026-07-25; OpenCL + CUDA-validated, multi-compartment GPU Hines solve)
C2	Permanent GitHub link to code/repository used for this code version	https://github.com/WarsawIQ/genesis-2.5 (archived release: Zenodo DOI <i>to be inserted upon release</i>)
C3	Legal Code License	GNU General Public License v2 or later (program); GNU Lesser General Public License v2.1 or later (library portions) – see LICENSE
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	C, OpenCL C, CUDA C++ (accelerator backends), MPI (PGENESIS), Python (benchmark analysis/plotting)
C6	Compilation requirements, operating environments & dependencies	Linux x86_64, GCC, GNU make, an OpenCL 1.2+ runtime (tested: ROCm 6.3.1 and Mesa rusticl) or a CUDA 12.x toolkit (tested: CUDA 12.8, sm_89 NVIDIA RTX 4090; sm_80 NVIDIA A100 and sm_86 NVIDIA A40 on the UMCS cluster) for the GPU backends, an MPI implementation for PGENESIS (MPICH/HYDRA or Open MPI tested on the UMCS cluster)
C7	If available Link to developer documentation/manual	README.md and paper/docs/REPLICATION.md in the repository
C8	Support email for questions	karol@chlasta.pl

Table 1: Code metadata (mandatory)

MOOSE) evolving into the CBI federation [4, 5, 6], not a single artifact comparable to GENESIS 2.4. GENESIS 2.5 is not a re-architecture and targets existing GENESIS 2.4 deployments that should not need to migrate models or scripts to gain accelerator support.

2. Software description

2.1 Software architecture

GENESIS 2.5 keeps the existing GENESIS/PGENESIS architecture (SLI interpreter, `hsolve` fast compartmental solver, PGENESIS MPI partitioning) unmodified and adds one new component: an OpenCL backend invoked from `hsolve` when an attached `hsolve` element uses `chanmode=4` (or 5) with real ion-channel state. Two dispatch modes are implemented in `ocl_hsolve.c` against `genesis/src/hines/opencl/ocl_channel.cl`:

- *Per-step dispatch* (`ocl_chip_update`): one `clEnqueueNDRangeKernel` call per simulation step, with channel state uploaded and downloaded every step.
- *Multiloop dispatch* (`chip_channel_multiloop`, via `GENESIS_OCL_MULTILOOP=<K>`): a single kernel dispatch runs all K steps in an inner GPU-side loop, amortizing the per-step PCIe transfer cost that otherwise dominates wall time (Section 3).

Device attribution and dispatch are confirmed at run time: the command queue uses `CL_QUEUE_PROFILING_ENABLE`, and every accelerator run emits a non-empty `OCL PROFILING SUMMARY`; this guards against an `hsolve` with no attached channels never reaching the kernel, and against a kernel that fails to build silently falling back to CPU. The channel kernel is built in `float` (fp32), so it runs on runtimes exposing no fp64 support (Mesa rusticl on the AMD Radeon 890M used here); the rest of `hsolve` remains `double`, with conversion only at the buffer boundary.

Both dispatch modes update each compartment independently, exact only for single-compartment (isopotential) neurons: a real dendritic tree's compartments are coupled through axial resistance, solved on the CPU by `hsolve` as a compiled bytecode program performing a bottom-up/top-down Gaussian elimination over the tree's tridiagonal system once per step. We add a third kernel, `hines_tree_eliminate` (`ocl_channel.cl`; CUDA port in `cuda/cuda_channel.cuh`), executing this identical bytecode on the GPU, one thread per disconnected tree (neuron), operating on the same flat opcode/coefficient arrays the CPU solver already builds at `SETUP` time. Because independent trees never reference each other's rows (the combined program is block-diagonal), threads need no synchronization. The channel and elimination kernels dispatch back to back on the same queue/stream, so the K -step batch still uploads and downloads state once. `ocl_hsolve.c/cuda_hsolve.c` select this path whenever the `hsolve` contains a multi-compartment tree, leaving the original kernel for single-

compartment networks.

2.2 Software functionalities

- Unmodified serial (`genesis`), headless (`nxgenesis`), and MPI (PGENESIS) execution paths.
- OpenCL channel-update kernel for Hodgkin–Huxley-style `tabchannel` gating ($\text{Na } X^3Y^1$, $\text{K } X^4$), dispatched per-step or batched via multiloop.
- A CUDA backend on the same call site (`genesis/src/hines/cuda/`): a line-for-line fp32 port of the OpenCL kernels behind an identical entry point, so the same models and harness drive either backend. Validated on an RTX 4090 and, independently, UMCS A100/A40 nodes (Section 3).
- Device-side dispatch confirmation (`OCL/CUDA_PROFILING_SUMMARY`) distinguishing a genuine GPU run from CPU fallback.
- A driven-spiking benchmark (`hh_spiking_benchmark.g`): N Hodgkin–Huxley neurons under one `hsolve`, driven suprathreshold to fire action-potential trains, exercising the repaired `inject` path.
- Three corrected Hines-solver defects (forward-elimination fast-path guard, backward-elimination root handling, `SET_DIAG/SKIP_DIAG` row selection) giving correct, independent per-neuron voltage dynamics under current injection, independent of GPU support.
- A GPU tridiagonal (Hines) elimination kernel, `hines_tree_eliminate` (OpenCL and CUDA), extending acceleration to real, axially-coupled multi-compartment dendritic trees (Section 3).
- Five corrected $O(n^2)$ model-construction defects in core GENESIS element-creation, path-resolution, and `hsolve` SETUP code, restoring linear-time construction independent of GPU support (Section 3).

3. Illustrative examples

Reproducing the CPU baseline. `nxgenesis` on the bundled `Ca18.g` and `Ca17difshell.g` models gives stable, low-variance timings (CV $\approx 1\%$, 30 replicates), confirming ordinary GENESIS workflows are unaffected by this release.

Reproducing the GPU multiloop result. `genesis/Scripts/benchmark/hh_spiking_benchmark.g` builds N driven, spiking Hodgkin–Huxley neurons under one `hsolve` (`chanmode=4`); `GENESIS_OCL_MULTILOOP=K` dispatches all K steps in one GPU call. Both arms (`nxgenesis_nocl`

fp64 CPU; `nxgenesis` fp32 multiloop, AMD Radeon 890M) are timed identically at $K = 50\,000$ and $K = 0$, isolating the *simulation phase* from construction; 30+ reps, 95% CIs in `experiments/data/campaign_wallclock_summary.csv`.

Table 2 and Fig. 1 report the result after the scheduler and solver fixes. Step-phase speedup peaks near $N = 5000$ at $\sim 18\times$ sustained and $\sim 13.5\times$ cold-start ($\sim 1.5 \times 10^9$ neuron-steps/s); the gap reflects the idle, downclocked GPU during construction. Beyond $N \approx 10\,000$ the integrated GPU no longer sustains its fast clock state and speedup falls toward $\sim 3\times$. End-to-end speedup peaks at $\sim 4\times$ and approaches the step-phase figure as simulations lengthen (Fig. 5).

N	CPU step (s)	GPU step (s)	Step-phase sustained	Step-phase single-run	Throughput (M nsteps/s)
500	0.302	0.112	2.7 $\times$	2.0 $\times$	223
1 000	0.587	0.112	5.2 $\times$	3.7 $\times$	446
2 000	1.166	0.117	10.0 $\times$	6.5 $\times$	855
5 000	2.965	0.164	18.1$\times$	13.5 $\times$	1524
10 000	5.799	0.406	14.3 $\times$	6.8 $\times$	1232
20 000	11.918	3.283	3.6 $\times$	3.1 $\times$	305
50 000	31.179	9.880	3.2 $\times$	3.2 $\times$	253

Table 2: GPU multiloop (fp32) vs. CPU (fp64) baseline, AMD Radeon 890M, driven spiking neurons, $K = 50\,000$ steps. “Sustained” is the warm steady-state dispatch; “single-run” is a cold-start run, $\sim 1.3\text{--}2\times$ slower. End-to-end (incl. construction) speedup is $1.2\text{--}4.2\times$. Raw data: `experiments/data/campaign_warm_dispatch.csv`, `experiments/data/campaign_wallclock_summary.csv`, `experiments/data/campaign_reproducibility.csv`.

Numerical fidelity of the fp32 accelerator. `genesis/Scripts/benchmark/hh1952_ap_verify.g` compares the classic 1952 squid AP across CPU fp64, GPU per-step, and GPU multiloop paths: fp32 agrees with fp64 to $\sim 2 \times 10^{-7}$ V over a single AP, all N neurons identical to $< 10^{-6}$ V (Fig. 2). Over long spiking runs, fp32/fp64 traces drift in spike *phase* but preserve waveform and firing rate, why timing uses driven neurons while correctness uses a single-AP window.

CUDA backend on NVIDIA hardware. The CUDA backend builds from the same source (`make USE_CUDA=1`) against the identical harness. On an NVIDIA RTX 4090 (Ada, CUDA 12.8, fp32) the multiloop kernel reproduces the fp64 CPU action potential to 7×10^{-8} V, confirming parity with both CPU and OpenCL paths. A 10-replicate sweep on all three devices (Table 3) rises with network size rather than

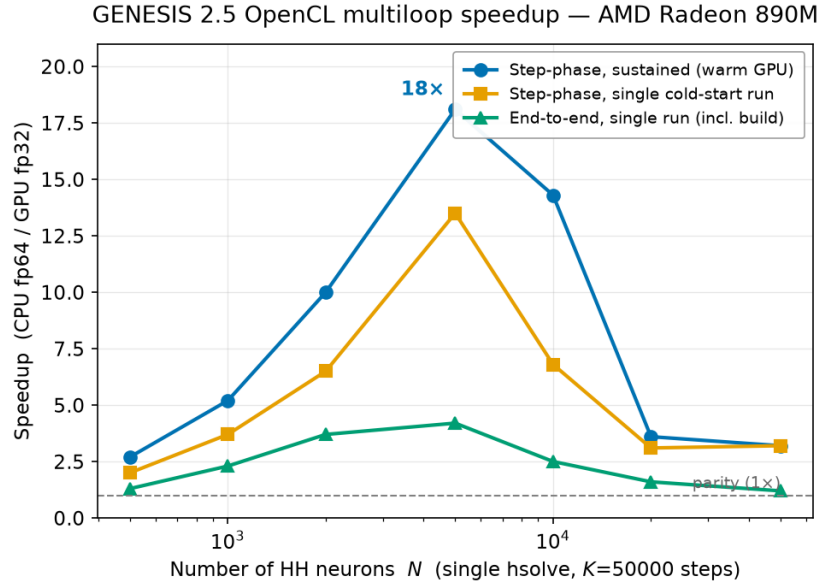


Figure 1: Measured wall-clock speedup, GPU multiloop (fp32) vs. CPU (fp64), sustained and cold-start step-phase, and end-to-end (single run). Both step-phase series peak near $N = 5000$ and fall beyond $N \approx 10000$ as the integrated GPU stops sustaining its fast clocks.

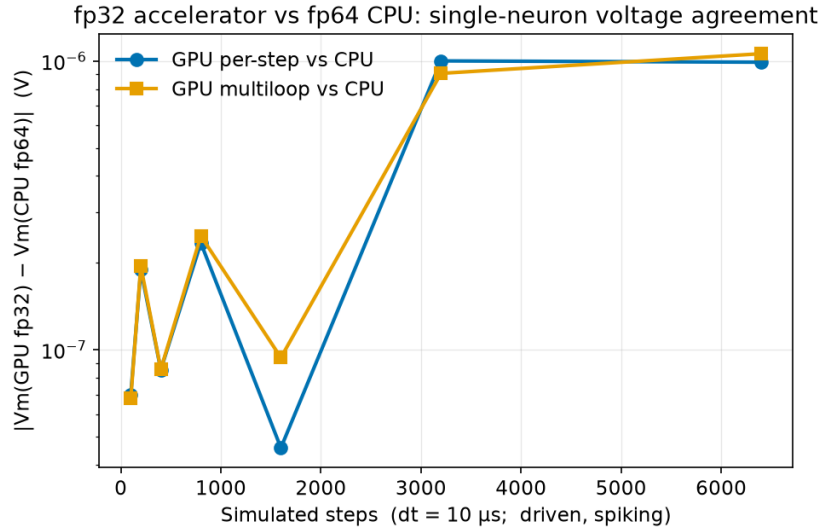


Figure 2: fp32 accelerator vs. fp64 CPU: absolute membrane-voltage difference for a single neuron as a function of simulated steps, for both the per-step and multiloop GPU paths.

peaking like the integrated GPU; A40 unexpectedly outperforms both A100 and RTX 4090 at every N ($212.7\times$ vs. $178.4\times$ and $174.2\times$ at

$N = 50\,000$), opposite the tree-elimination kernel’s ordering (Table 4), plausibly since this dispatch is lightweight and latency-bound.

N	RTX 4090 speedup (mean $\pm$ std, 10 reps)	A40 speedup (mean $\pm$ std, 10 reps)	A100 speedup (mean $\pm$ std, 10 reps)
500	$7.4 \pm 0.17\times$	$7.6 \pm 0.02\times$	$5.9 \pm 1.5\times$
5 000	$70.3 \pm 0.06\times$	$75.1 \pm 0.1\times$	$34.3 \pm 0.4\times$
50 000	$174.2 \pm 1.6\times$	$212.7 \pm 0.6\times$	$178.4 \pm 14.4\times$

Table 3: CUDA backend (fp32) vs. CPU (fp64) single-compartment multiloop kernel (`hh_spiking_benchmark.g`, $K = 50\,000$ steps), mean $\pm$ std, 10 replicates. RTX 4090: WarsawIQ. A40/A100: UMCS cluster. GPU arm timed via the device-side CUDA MULTILoop report, not the script’s {cpu} timer (≈ 0 for this arm). **A40 outperforms both A100 and RTX 4090 at every N here**, opposite the multi-compartment kernel (Table 4). Raw data: `cluster_bringup/logs/cuda_singlecomp_campaign_wiq RTX4090_20260727_210047.csv` (RTX 4090), `..._inf02_A40_20260727_024553.csv` (A40), `..._inf03_A100_20260727_024613.csv` (A100).

Multi-compartment GPU acceleration. The multiloop kernel above is exact only for single-compartment (isopotential) neurons: for a real dendritic tree it silently drops all axial coupling. `hines_tree_eliminate` (Section 2.1) performs this elimination on-device, dispatched whenever the `hsolve` contains a multi-compartment tree.

Correctness. `hh_multicompartment_benchmark.g` (linear-chain) and `hh_branching_multicompartment_benchmark.g` (branching, exercising sibling-array opcodes) compare GPU (chanmode 4) against CPU (chanmode 1, fp64) across $N_{\text{COMP}} = 1\text{--}32$, up to 4 branches, and up to $N = 10\,000$ neurons: agreement is $\sim 10^{-7}$ V, matching single-compartment fidelity. One narrow topology, a single-compartment branch directly off the soma in the first tree built, is excluded and flagged explicitly (`genesis/src/hines/GPU_HINES_SOLVE_DESIGN.md`); such zero-length stubs do not occur in practice.

Speed. Table 4 and Fig. 3 report step-phase speedup (mean $\pm$ std, 10 replicates), sweeping $N = 100\text{--}50\,000$ at $N_{\text{COMP}} = 16$, a range unreachable before the model-construction fix (Impact): $N = 50\,000$ took 23.8 ± 0.3 s to build there, versus not finishing before. Measured on UMCS NVIDIA A40/A100, WarsawIQ NVIDIA RTX 4090, and, for $N \leq 1200$, WarsawIQ AMD Radeon 890M, whose safety cap (Section 2.1) was overridden to confirm the previously-documented hang threshold at $N = 1500$. Branching topology gives statistically indistinguishable speedups and is omitted from the figure. Dispatch uses $K = 200$ steps ($K = 100$ for $N \geq 5000$), smaller than the single-compartment sweeps’ $K = 50\,000$. All four devices are slower

than CPU at $N \leq 500$; past $N \approx 500$ –1000 the GPUs pull ahead, reaching $37.3 \pm 0.2\times$ (A40), $82.1 \pm 1.1\times$ (A100), and $60.7 \pm 1.0\times$ (RTX 4090) at $N = 50\,000$, none plateaued. A100 leads throughout, ahead of RTX 4090 then A40, the opposite ordering from the single-compartment kernel (Table 3). Raw data: `cluster_bringup/logs/weekend_campaign_inf02_A40_20260725_212053.csv`, `..._inf03_A100_20260725_212113.csv`, `..._wiq_RTX4090_20260727_194140.csv`, `..._local_AMD890M_20260725.csv`.

Parallelization granularity. `hines_tree_eliminate` assigns one GPU thread per tree, so speedup comes from spreading many neurons across threads, not from parallelizing elimination *within* a tree. For a single detailed morphology alone ($N = 1$), only one thread is active: measured directly (AMD Radeon 890M), the kernel is ~ 20 – $21\times$ slower than the CPU at $\text{NCOMP} = 2000$ and 8000 , stable with tree size as expected for a single-thread bottleneck. The kernel therefore accelerates population-scale simulation (Table 4), not the single-detailed-cell regime.

N	total comps	A40	A100	RTX 4090	AMD 890M
100	1 600	0.2 ± 0.02	0.3 ± 0.01	0.1 ± 0.01	0.2 ± 0.01
500	8 000	1.0 ± 0.03	1.7 ± 0.11	0.5 ± 0.02	0.5 ± 0.05
1 000	16 000	1.9 ± 0.05	3.1 ± 0.09	1.1 ± 0.05	1.9 ± 0.17
1 200	19 200	–	–	–	2.8 ± 0.87
2 000	32 000	4.4 ± 0.43	6.4 ± 0.23	2.6 ± 0.09	–
3 000	48 000	6.7 ± 0.04	12.4 ± 0.26	6.8 ± 0.20	–
5 000	80 000	6.8 ± 0.03	15.1 ± 0.33	8.2 ± 0.24	–
7 000	112 000	9.7 ± 0.06	22.2 ± 0.30	11.5 ± 0.34	–
10 000	160 000	13.6 ± 0.08	30.3 ± 0.76	16.5 ± 0.40	–
15 000	240 000	20.1 ± 0.13	41.9 ± 0.95	23.8 ± 0.36	–
20 000	320 000	25.2 ± 0.11	51.7 ± 1.05	30.9 ± 0.48	–
30 000	480 000	32.0 ± 0.19	66.8 ± 1.23	43.0 ± 1.04	–
40 000	640 000	35.2 ± 0.17	76.6 ± 0.59	52.3 ± 0.95	–
50 000	800 000	37.3 ± 0.19	82.1 ± 1.09	60.7 ± 1.01	–

Table 4: Multi-compartment GPU tree-elimination (fp32) vs. CPU (fp64), step-phase speedup (mean $\pm$ std, 10 replicates), UMCS A40/A100, WarsawIQ RTX 4090, and local WarsawIQ AMD Radeon 890M, $\text{NCOMP} = 16$, linear-chain, batched multiloop ($K = 200$, $K = 100$ for $N \geq 5000$). AMD 890M stops at $N = 1200$; $N = 1500$ hangs the iGPU driver (see text). “–” = not run, not zero. Total sweep time: 116.8 min (A40), 119.8 min (A100), 79 min (RTX 4090), ~ 1 min (AMD 890M). Raw data: `cluster_bringup/logs/weekend_campaign_inf02_A40_20260725_212053.csv` (A40), `..._inf03_A100_20260725_212113.csv` (A100), `..._wiq_RTX4090_20260727_194140.csv` (RTX 4090), `..._local_AMD890M_20260725.csv` (AMD 890M).

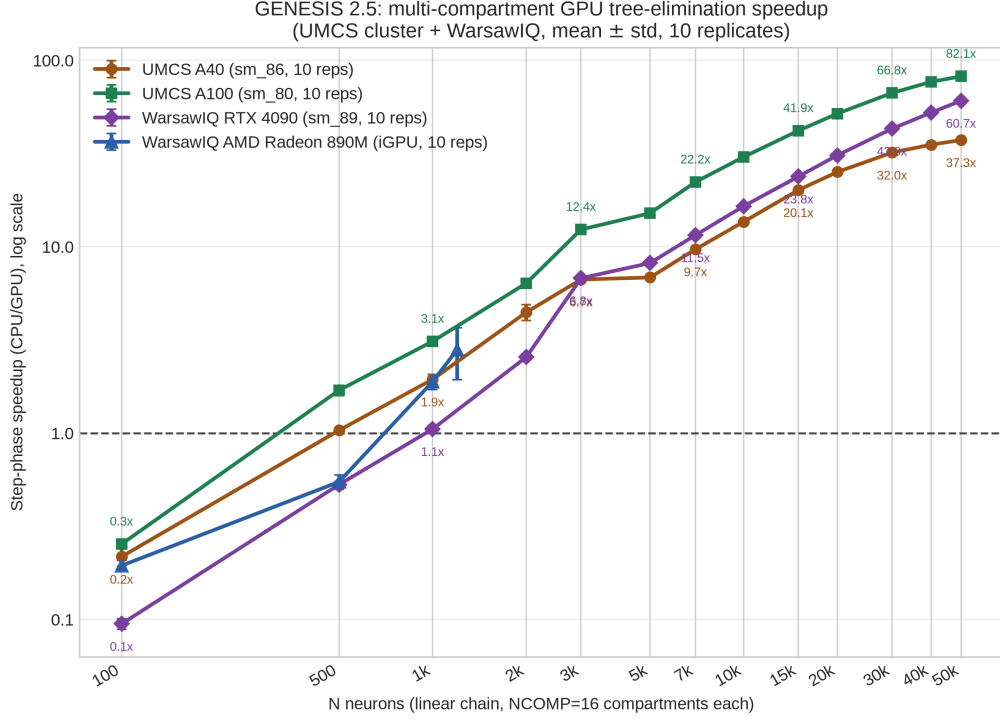


Figure 3: Multi-compartment GPU tree-elimination step-phase speedup vs. CPU (mean $\pm$ std, 10 replicates; log-log), UMCS A40/A100, WarsawIQ RTX 4090, plus local WarsawIQ AMD Radeon 890M, sweeping N at $\text{NCOMP} = 16$ (linear-chain). All four are slower than CPU at small N and grow past a crossover near $N \approx 500$ – 1000 ; A100 leads throughout, reaching $82.1\times$ at $N = 50\,000$, ahead of RTX 4090 ($60.7\times$) and A40 ($37.3\times$). The AMD 890M series stops at $N = 1200$ (19 200 compartments); $N = 1500$ reproducibly hangs the iGPU driver (Section 2.1).

Population-scale model construction was $O(n^2)$; now $O(n)$. Pushing the multi-compartment benchmark toward the $\sim 31\,000$ -neuron Blue Brain Project scale [9] exposed a defect independent of GPU support: building $N = 31\,000$ 17-compartment neurons (527 000 compartments) did not finish within 900 s, though Table 4 only required 160 000. Profiling found a fitted exponent of 2.31 ($O(n^2)$), traced to five pre-existing linear-scan-per-element patterns in core GENESIS code (element creation, path resolution, `hsolve` SETUP), none GPU-specific: each rescanned siblings, paths, or compartments in $O(n)$ where hashing or amortized growth would do. We fixed all five with hash-based lookups, load-factor rehashing, and geometric growth. Table 5 and Fig. 4 show a fitted exponent of 0.99 after, effectively linear, extended to $N = 100\,000$ (1.7 million compartments): the 527 000-compartment population now builds in 14.6 ± 0.2 s, down from not

finishing, and the 1.7-million-compartment population in 51.5 ± 0.6 s. Changes were verified to produce byte-identical output on every validation script, confirming a pure performance fix; full diagnosis in `genesis/src/hines/GPU_HINES_SOLVE_DESIGN.md`.

N	total comps	before (s)	after: mean $\pm$ std (s)	CV
1 000	17 000	0.82	0.57 ± 0.08	13.2%
2 000	34 000	2.03	0.93 ± 0.003	0.3%
4 000	68 000	8.24	1.83 ± 0.04	1.9%
8 000	136 000	105.45	4.08 ± 0.03	0.7%
16 000	272 000	did not finish (60 s+)	7.54 ± 0.02	0.2%
31 000	527 000	did not finish (900 s+)	14.57 ± 0.18	1.2%
50 000	850 000	–	23.80 ± 0.32	1.3%
70 000	1 190 000	–	34.25 ± 0.44	1.3%
100 000	1 700 000	–	51.48 ± 0.61	1.2%

Table 5: Model-construction wall time (construction + `hsolve SETUP` + one step), `hh_branching_multicompartment_benchmark.g`, local machine, before vs. after the five fixes. “After”: mean $\pm$ std, 3 replicates; “before”: single replicate, not re-measured beyond $N = 8000$ (“–” = never attempted). Fitted exponent: 2.31 before, 0.99 after. Raw data: `experiments/data/construction_scaling_before_after.csv`.

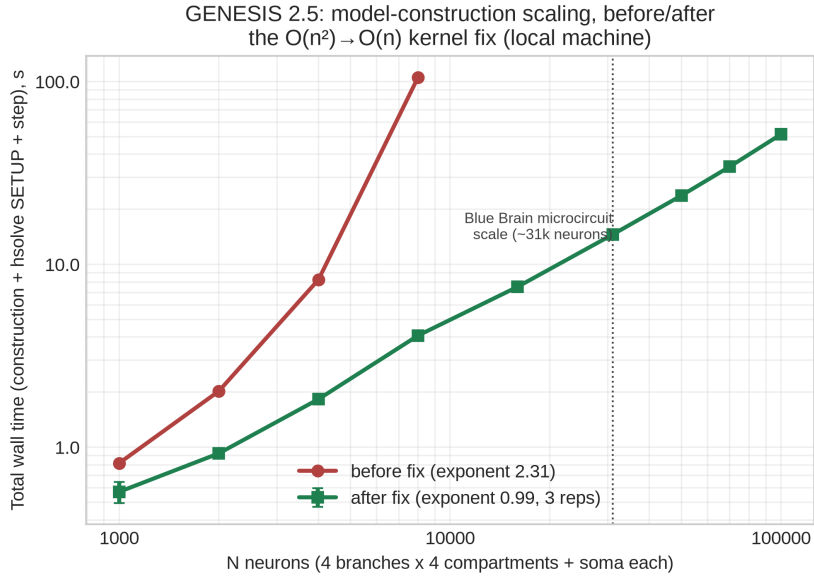


Figure 4: Model-construction wall time vs. N , log-log, before (single replicate) and after (mean $\pm$ std, 3 replicates) the five $O(n^2)$ fixes, extended to $N = 100\,000$. The “before” series stops at $N = 8000$: larger N did not finish within the tested budget.

Speedup grows with simulation length. Model construction is a fixed one-time cost, so end-to-end speedup depends on run length. Sweeping K at fixed $N = 5000$ (Fig. 5) shows end-to-end speedup rising toward the step-phase ceiling: the accelerator pays off most for the long runs typical of real modeling studies (`experiments/data/campaign_ksweep.csv`).

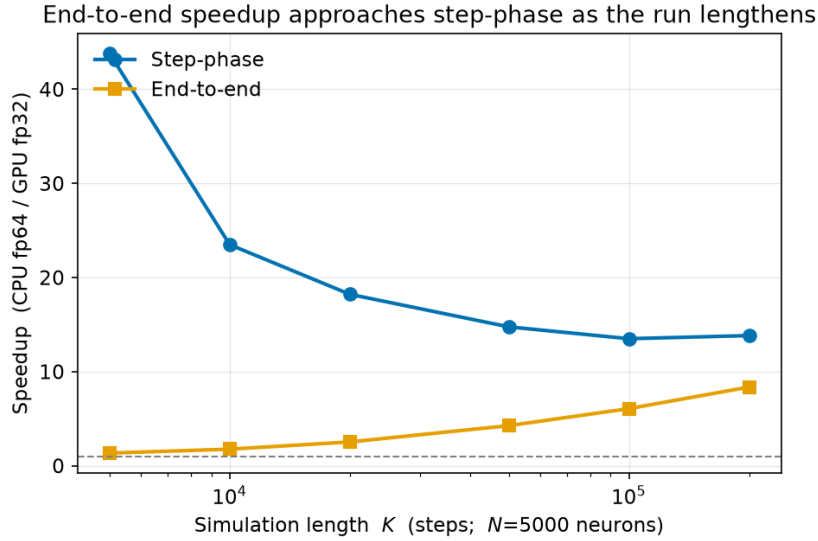


Figure 5: Speedup vs. simulation length K at fixed $N = 5000$. As the run lengthens, the one-time model-construction cost amortizes and the end-to-end speedup approaches the step-phase (simulation-only) speedup.

Reproducing the PGENESIS scaling result. The existing MPI path is unaffected; a strong-scaling sweep ($N = 2400$ HH neurons, $P = 1$ –24 ranks, 12-core/24-thread host) gives near-ideal scaling at $P = 2$ ($E_P = 0.95$), an efficiency knee near $P \approx 5$, and a recommended $P = 4$ –8 (3.1 – $4.0\times$, 50–78% efficiency) for this problem size.

Full replication steps and raw data are in `paper/docs/REPLICATION.md` and the self-contained `experiments/` package.

4. Impact

GENESIS/PGENESIS users with channel-kinetics-heavy models now have a path to GPU acceleration (Section 3) without migrating models, SLI scripts, or MPI workflows. That is the principal barrier an architecturally different successor such as MOOSE would otherwise impose. The three Hines-solver defects fixed along the way (Section 2) are independent of GPU support and affect any multi-neuron `hsolve` configuration: before the fix, `inject`-driven voltage transients were

silently suppressed for every neuron but the first, with no error raised. That defect predates this release and is now fixed for any multi-neuron `hsolve` user, whether or not they use an accelerator.

Bringing up the accelerator backends surfaced four confounds: a GUI-linked binary, a benchmark that never dispatches to the kernel, CPU-timer semantics that overstate GPU-blocked wait as compute, and an off-by-one `argc` guard. Each produced a false reading at some point, and the wall-clock protocol built to catch them should generalize to any future GENESIS accelerator backend. On the UMCS cluster this scaled to 10-replicate statistical rigor for both kernels (Tables 3, 4), the multi-generation GPU comparison the cluster access was meant to provide. The dominant early cost was a broken $O(N)$ scheduler loop leaving neurons $2 \dots N$ unabsorbed and re-scheduled every step; fixing it is what makes Section 3’s numbers possible. A further, structural limit remained: the multiloop kernel updates each compartment independently, so it only works for single-compartment networks. `hines_tree_eliminate` (Section 2.1, Section 3) closes that gap the way Arbor [7] and CoreNEURON [8] do for other simulators, but not the single-detailed-cell case, which needs intra-tree parallelism, left for future work.

Pushing toward the Blue Brain Project’s scale [9] turned up a defect entirely separate from the GPU work: five pre-existing linear-scan patterns compounding into $O(n^2)$ model-construction time (Section 3). None is accelerator-specific, so fixing all five benefits any GENESIS/PGENESIS user building a large model: a second latent defect found only because this work pushed the codebase to a scale it had apparently never seen before.

5. Conclusions

GENESIS 2.5 adds validated, opt-in OpenCL and CUDA acceleration to GENESIS 2.4/PGENESIS 2.4 while keeping every existing CPU, MPI, model, and SLI-script workflow intact (Section 3 reports full speedup and fidelity numbers for both integrated-GPU and cluster-class hardware). Three latent Hines-solver defects affecting any multi-neuron `hsolve` configuration are fixed independent of accelerator use. Beyond the original single-compartment multiloop kernel, `hines_tree_eliminate` closes the gap for real, axially-coupled multi-compartment dendritic trees: population-scale acceleration, not the single-detailed-cell case, which this design does not help and which we measure and report explicitly rather than omit. Pushing this population toward a literature-relevant scale [9] surfaced a further, GPU-independent defect: five pre-existing linear-scan patterns compounding into $O(n^2)$ model-construction

time; fixing all five restores linear scaling and benefits any GENESIS/PGENESIS user building a large model, independent of accelerator use. Recorded next steps: intra-tree parallelism (e.g. cyclic reduction) to accelerate the single-detailed-cell regime; end-to-end validation on established PGENESIS network models, such as liquid-state-machine reservoirs, to confirm the unmodified-model claim and characterize combined MPI+GPU scaling; a head-to-head benchmark against other GPU-capable compartmental simulators (NEURON/CoreNEURON [8], Arbor [7]) on an equivalent model, which this work does not attempt; and packaging this release into a container-based on-premise/cloud deployment pipeline [10].

Acknowledgements

The authors thank the Maria Curie-Skłodowska University (UMCS) in Lublin and the LubMAN UMCS computing centre for access to the “Lunar” cluster (NVIDIA A100 and A40 GPU nodes) at the UMCS Institute of Mathematics, commissioned in 2015 [11] and since upgraded with the GPU nodes used in this work, and Karol S. Karpowicz and Prof. Przemysław Stpiczyński for provisioning and support of the compute environment on which the CUDA backend was built and validated. The authors also thank WarsawIQ for the AMD Radeon 890M (integrated GPU) and NVIDIA RTX 4090 hardware used for the laptop-class and desktop-class benchmarks reported here.

References

- [1] Bower JM, Beeman D. *The Book of GENESIS: Exploring Realistic Neural Models with the GEneral NEural Simulation System*. Springer, 1998.
- [2] GENESIS 3 Development Plans. <http://www.genesis-sim.org/GENESIS/G3/G3-dev-plans.html>
- [3] Cornelis H, Coop AD, Rodriguez AL, Beeman D, Bower JM. GENESIS 3.0 functionality enhanced by interface to multiple types of database. *BMC Neuroscience* 2011, 12(Suppl 1):P15.
- [4] Cornelis H, Edwards M, Coop AD, Bower JM. The CBI architecture for computational simulation of realistic neurons and circuits in the GENESIS 3 software federation. *BMC Neuroscience* 2008, 9(Suppl 1):P88.
- [5] Cornelis H, Lu H, Esquivel A, Bower JM. Modeling a single dendritic compartment using Neurospaces and GENESIS-3. *BMC Neuroscience* 2007, 8(Suppl 2):P3.

- [6] Cornelis H, Bampasakis D, Steuber V, Bower JM. Interoperability in the GENESIS 3.0 Software Federation: the NEURON Simulator as an Example. *BMC Neuroscience* 2013, 14(Suppl 1):P33.
- [7] Akar NA, Cumming B, Karakasis V, Küsters A, Klijn W, Peyser A, Yates S. Arbor – A Morphologically-Detailed Neural Network Simulation Library for Contemporary High-Performance Computing Architectures. In: *27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, 2019, pp. 274–282. doi:10.1109/EMPDP.2019.8671560.
- [8] Kumbhar P, Hines M, Fouriaux J, Ovcharenko A, King J, Delalondre F, Schürmann F. CoreNEURON: An Optimized Compute Engine for the NEURON Simulator. *Frontiers in Neuroinformatics* 2019, 13:63. doi:10.3389/fninf.2019.00063.
- [9] Markram H, Muller E, Ramaswamy S, et al. Reconstruction and Simulation of Neocortical Microcircuitry. *Cell* 2015, 163(2):456–492. doi:10.1016/j.cell.2015.09.029.
- [10] Chlasta K, Sochaczewski P, Wójcik GM, Krejtz I. Neural simulation pipeline: Enabling container-based simulations on-premise and in public clouds. *Frontiers in Neuroinformatics* 2023, 17:1122470. doi:10.3389/fninf.2023.1122470.
- [11] Maria Curie-Skłodowska University (UMCS), Instytut Matematyki. Superkomputer LUNAR uruchomiony w Instytucie Matematyki UMCS. <https://www.umcs.pl/pl/aktualnosci,57,superkomputer-lunar-uruchomiony-w-instytucie-matematyki-umcs,27879.htm>, 2015.

Current executable software version

Not applicable; this release is distributed as source code only (see Current code version above).